

HolaMundo con Hibernate – parte 2



Guía: Crear clase de entidad y ejecutar una consulta con Hibernate – parte 2

Introducción

En esta lección daremos continuidad al proyecto creado previamente y realizaremos los siguientes pasos:

- ♦ Crear la clase de entidad `Persona.java`.
- ♦ Escribir la clase de prueba `HolaMundoHibernate.java` para ejecutar una consulta HQL.
- ♦ Realizar pruebas y resolver errores comunes de conexión.

Paso 1: Crear la clase de entidad `Persona`

Ruta del archivo:

HibernateEjemplo1/src/main/java/gm/domain/Persona.java

Código:

```
package gm.domain;

import java.io.Serializable;
import jakarta.persistence.*;

@Entity
@Table(name = "persona")
public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;

    @Column(name="id_persona")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Id
    private Integer idPersona;

    private String nombre;

    private String apellido;

    private String email;

    private String telefono;

    public Persona(){}

    public Integer getIdPersona() {
        return idPersona;
    }

    public void setIdPersona(Integer idPersona) {
        this.idPersona = idPersona;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
```

```
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido +
            ", email=" + email + ", telefono=" + telefono + '}';
    }
}
```

Explicación de la clase `Persona`

Esta clase representa una entidad **JPA** (Jakarta Persistence API) que se va a mapear a la tabla `persona` en la base de datos. Vamos a explicar los puntos clave:

```
@Entity
@Table(name = "persona")
public class Persona implements Serializable {
```

- `@Entity`: indica que esta clase será persistida en una base de datos.
- `@Table(name = "persona")`: mapea la clase a la tabla llamada `persona`.
- `implements Serializable`: es una buena práctica para entidades JPA, permite que los objetos puedan ser enviados a través de streams, por ejemplo, en aplicaciones distribuidas.

🔑 Llave primaria con `Integer` y autoincremento

```
@Column(name="id_persona")
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Id
private Integer idPersona;
```

- `@Id`: indica que este campo es la llave primaria.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: Hibernate delega a la base de datos la generación automática del valor del ID (ej. MySQL con `AUTO_INCREMENT`).
- `@Column(name="id_persona")`: mapea el atributo al campo `id_persona` de la tabla.

! ¿Por qué usamos `Integer` en lugar de `int`?

- Si usamos `int` (tipo primitivo), su valor **por defecto es 0**.
- Hibernate **interpreta el valor 0 como si tú ya le asignaste manualmente un ID, y no genera uno nuevo automáticamente**, provocando errores como:

```
IdentifierGenerationException: Identifier of entity must be manually assigned
```

- En cambio, con `Integer` (tipo objeto), el valor por defecto es `null`, y Hibernate entiende que **debe generar el valor automáticamente** desde la base de datos.

✅ Por eso **se recomienda usar `Integer` o `Long` en lugar de `int` o `long`** para las llaves primarias generadas automáticamente.

2 Crear el archivo `log4j2.properties` para mostrar los parámetros SQL

Creamos un archivo llamado `log4j2.properties` en la ruta `src/main/resources`. Este archivo permite configurar el **nivel de detalle del log**, especialmente útil para **ver los parámetros** que se envían en las consultas SQL.

📄 Contenido del archivo `log4j2.properties`:

```
# SQL ejecucion
logger.hibernate.name = org.hibernate.SQL
logger.hibernate.level = debug
```

```
# JDBC parametros binding
logger.jdbc-bind.name=org.hibernate.orm.jdbc.bind
logger.jdbc-bind.level=trace
```

🔗 ¿Qué hace cada línea?

- `logger.hibernate.name = org.hibernate.SQL`: activa el log para mostrar las consultas SQL que ejecuta Hibernate.
- `logger.hibernate.level = debug`: establece el nivel de log en **debug** para mostrar las sentencias SQL.
- `logger.jdbc-bind.name = org.hibernate.orm.jdbc.bind`: activa el log para mostrar los **valores concretos** que se están enviando como parámetros en las sentencias SQL.
- `logger.jdbc-bind.level = trace`: establece el nivel de log en **trace**, lo cual permite ver cada parámetro enlazado (bound) a la consulta SQL.

✅ Con esta configuración, podrás ver en consola salidas como:

```
DEBUG org.hibernate.SQL - insert into persona (apellido,email,nombre,telefono) values
(?,?,?,?)
TRACE org.hibernate.orm.jdbc.bind - binding parameter (1:VARCHAR) <- [Lara]
TRACE org.hibernate.orm.jdbc.bind - binding parameter (2:VARCHAR) <- [clara@mail
```



Paso 3: Crear la clase de prueba `HolaMundoHibernate.java`



Ruta del archivo:

`HibernateEjemplo1/src/main/java/test/HolaMundoHibernate.java`

Código:

```
package test;

import gm.domain.Persona;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class HolaMundoHibernate {
    public static void main(String[] args) {
        String hql = "SELECT p FROM Persona p";

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("HibernateEjemplo1");

        try (EntityManager em = emf.createEntityManager()) {
            var personas = em.createQuery(hql, Persona.class).getResultList();
        }
    }
}
```

```

        personas.forEach(persona -> System.out.println(" " + persona));
    }
}
}

```

✚ Explicación del código:

- Se importa la clase `Persona`, que es una entidad JPA mapeada a la tabla `persona` en la base de datos.
- Se crea una cadena con el lenguaje HQL (Hibernate Query Language): `"SELECT p FROM Persona p"` para seleccionar todas las personas.
- Se crea una `EntityManagerFactory` a partir del nombre de la unidad de persistencia definida en `persistence.xml` (`HibernateEjemplo1`).
- Dentro del bloque `try-with-resources`, se crea una instancia de `EntityManager`, lo cual permite gestionar operaciones con la base de datos.
- Se ejecuta la consulta con `createQuery`, especificando el tipo de resultado `Persona.class`, y se obtienen los resultados en una lista.
- Finalmente, se recorre la lista e imprime cada objeto `Persona` usando su método `toString()`.

✅ Este ejemplo comprueba que Hibernate está configurado correctamente y que puede recuperar registros desde la base de datos.



Paso 3: Ejecutar la clase de prueba

Haz clic derecho sobre `HolaMundoHibernate.java` → **Run File**

✅ Si todo está configurado correctamente, verás el listado de personas impreso en consola, junto con los `SELECT` generados por Hibernate.

```
Hibernate: select p1_0.id_persona,p1_0.apellido,p1_0.email,p1_0.nombre,p1_0.telefono from persona p1_0
```

```
Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
```

```
Persona{idPersona=2, nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}
```

```
Persona{idPersona=3, nombre=Maria, apellido=Esparza, email=mesparza@mail.com, telefono=55112233}
```

```
Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}
```



Script SQL para crear la tabla en MySQL

En caso de que no tengas creada la base de datos de `test`, ni la tabla de `persona`, aquí te dejamos un script de `MySQL` para que puedas crearla:

```
CREATE DATABASE IF NOT EXISTS test;
USE test;

DROP TABLE IF EXISTS persona;

CREATE TABLE persona (
    id_persona INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(45) NOT NULL,
    apellido VARCHAR(45) NOT NULL,
    email VARCHAR(100) UNIQUE,
    telefono VARCHAR(20),
    PRIMARY KEY (id_persona)
);

INSERT INTO persona (nombre, apellido, email, telefono) VALUES
('Rafael', 'Juarez', 'rjuarez@mail.com', '55667788'),
('Ivonne', 'Lara', 'ilara@mail.com', '55443322'),
('Maria', 'Esparza', 'mesparza@mail.com', '55112233'),
('Hugo', 'Hernandez', 'hhernandez@mail.com', '55778822');
```



Conclusión

En esta lección:

- ✓ Creamos la clase de entidad `Persona.java`.
- ✓ Escribimos una clase de prueba para ejecutar consultas con HQL.

Con este proyecto base listo, ya puedes comenzar a construir aplicaciones más complejas con Hibernate y JPA.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)